Devups

Tome II — Les données

Documentation du framework

Première édition — novembre 2026

Établie à partir du code réellement exécuté dans le projet **3agReader** (socle Devups 4.3).
Chaque affirmation porte sa source : *fichier:ligne*.

Sommaire

Chapitre 1 — Le datacore 5

1.1 Le principe 5
1.2 Le datacore d'un composant 5
1.3 Le datacore d'un module 5
1.4 Le datacore d'une entité 6
1.5 Les champs automatiques 7
1.6 Les trois choses que le datacore ne sait pas dire 8

Chapitre 2 — Les entités 9

2.1 La chaîne d'héritage 9
2.2 L'anatomie d'une entité 10
2.3 L'annotation de référence 11
2.4 Les propriétés dv* 11

Chapitre 3 — Les trois classes de la couche d'accès 13

3.1 L'architecture 13
3.2 Doctrine, et ce qu'il ne fait pas 13
3.3 La connexion partagée 13

Chapitre 4 — Lire 15

4.1 Les points d'entrée statiques 15
4.2 La clause WHERE 16
4.3 Tri, regroupement, limite 18
4.4 Les relations 18
4.5 Les méthodes terminales, et le piège null contre entité vide 19
4.6 Prévisualiser une requête sans l'exécuter 20
4.7 Exemples prêts à reprendre 21
4.8 Qualifier les colonnes : un défaut à connaître 22

Chapitre 5 — Écrire 23

5.1 Les formes 23
5.2 Deux pièges d'écriture 23
5.3 Les transactions 24
5.4 Les accesseurs et le traitement de formulaire 24
5.5 Sérialisation 25

Chapitre 6 — Les classes de base d'entité du socle 26

6.1 ScanCore — les œuvres numérisées 26
6.2 OrderCore — les commandes 26
6.3 UserCore — les utilisateurs 27

Chapitre 7 — La traduction (dvtranslate) 28

7.1 Le modèle 28

7.2 La bonne séparation 28
7.3 Forcer une langue 29
7.4 Le défaut D15 — les lignes invisibles 29
7.5 La bonne pratique, quand on doit lire une colonne non traduite 32
7.6 Un second défaut, dans la même famille 32

Chapitre 8 — Pièges et vulnérabilités de la couche de données 33

Ce tome suppose le Tome I lu, en particulier le chapitre 7 (le scan du disque) et le chapitre 6 (la ligne de commande).

Chapitre 1 — Le datacore

1.1 Le principe

Dans Devups, la structure des données est décrite par des fichiers JSON appelés **datacores**. Ils ne sont pas lus à l'exécution pour construire les requêtes : ils servent au **générateur** et au **scan** qui établit la liste des composants, modules et entités du projet.

La vérité d'exécution, elle, est dans les **annotations** des classes d'entité, lues par Doctrine. Le datacore est l'intention ; l'annotation est le contrat.

> **Corollaire à retenir.** Quand les deux divergent — et ils divergent, voir §1.6 — c'est l'annotation qui gagne, parce que c'est elle que Doctrine lit pour construire la table. Le datacore ne redevient la vérité qu'au moment où l'on régénère l'entité, et il écrase alors tout ce qui a été ajouté à la main.

1.2 Le datacore d'un composant

`src/<composant>/<composant>Core.json` :

```
{
  "name": "monprojet",
  "template": "adminv2",
  "listmodule": [
    {
      "name": "ModuleTask",
      "project": "monprojet",
      "listentity": [ { "name": "task", "attribut": {}, "relation": [] } ]
    }
  ]
}
```

Clé	Rôle
`name`	le nom du composant ; doit correspondre au nom du dossier
`template`	le gabarit de back-office
`listmodule`	**ignoré** — écrasé par le scan des sous-dossiers (`Core.php:91`)

La présence de ce fichier est la **condition d'existence** du composant : `findprojectcore()` retourne `[]` sans lui, et le composant entier disparaît du framework.

1.3 Le datacore d'un module

`src/<composant>/<Module>/<module en minuscules>Core.json` :

```
{
  "name": "ModuleTask",
  "label": "Tâches",
  "icon": "fas fa-fw fa-check-square"
}
```

Clé	Lu par	Effet
`name`	`Core::updateDvupsTable()`	la clé de `module_configurations.php`
`label`	`Core.php:264`	le libellé dans la barre latérale du back-office ; par défaut, le nom du module
`icon`	`Core.php:265`	la classe d'icône ; par défaut `fas fa-fw fa-cog`

`{"name":"ModuleXxx"}` suffit à faire exister le module. Les deux autres clés ne servent qu'à l'affichage.

1.4 Le datacore d'une entité

`src/<composant>/<Module>/Core/<Entité>Core.json` :

```
{
  "name": "task",
  "label": "Tâche",
  "lang": ["name", "description"],
  "attribut": {
    "1": {
      "name": "name",
      "datatype": "string",
      "size": "255",
      "nullable": "not",
      "formtype": "text"
    }
  },
  "relation": [
    {
      "entity": "project",
      "cardinality": "manyToOne",
      "nullable": "default",
      "ondelete": "cascade"
    }
  ],
  "methods": []
}
```

Les cinq clés réellement lues par le générateur et le scan sont **`name`**, **`label`**, **`attribut`**, **`relation`**, **`lang`** (plus `methods`). Toute autre clé est inerte : sur 3agReader, `TreeCore.json` portait `module` et `projectid`, vestiges d'une version antérieure, retirés après vérification qu'aucun code ne les lit.

Le bloc `attribut`

C'est un **objet indexé par des chaînes numériques**, pas un tableau :

```
"attribut": {
  "1": { "name": "...", "datatype": "...", "size": "...", "nullable": "...", "formtype":
"..." },
  "2": { ... }
}
```

Clé	Valeurs	Note
`name`	minuscules, alphanumériques et `_`	devient le nom de colonne
`datatype`	`string`, `text`, `integer`, `float`, `date`, `datetime`, `json`, `boolean`	devient `@Column(type=...)`
`size`	entier en chaîne	pertinent pour `string`
`nullable`	`"not"` (NOT NULL) ou `"default"` (nullable)	**ignoré pour une relation** — voir §1.6
`formtype`	voir le tableau du Tome III	pilote le champ généré dans le formulaire

`id` est implicite. Il ne doit jamais figurer dans `attribut` : le générateur l'ajoute avec `@Id @GeneratedValue`.

Le bloc `relation`

C'est un **tableau** :

```
"relation": [
  { "entity": "project", "cardinality": "manyToOne", "nullable": "default", "ondelete": "cascade" }
]
```

Clé	Valeurs
`entity`	le nom en minuscules de l'entité cible
`cardinality`	`manyToOne`, `oneToMany`, `manyToMany`, `oneToOne`
`nullable`	**ignoré** (§1.6)
`ondelete`	`cascade`, `set null`, `restrict` — le seul attribut de `@JoinColumn` réellement interpolé

Le bloc `lang`

La liste des colonnes traduites. Sa présence fait de l'entité une entité **traduite** : le générateur produit en plus une classe `<Entité>_lang.php` et une table `<entité>_lang`. Voir le chapitre 7 de ce tome.

1.5 Les champs automatiques

Toute entité mappée reçoit `created_at`, `updated_at`, `deleted_at`. Ils ne se déclarent pas dans `attribut`.

`deleted_at` est le support de la suppression douce : `Model::$dvsoftdelete` à `true` fait que `__delete()` écrit une date au lieu de supprimer la ligne, et `QueryBuilder::handlesoftdelete()` filtre les lignes supprimées de toute lecture.

Les migrations SQL produites par `schema:update` sont archivées dans `database/migration/`.

1.6 Les trois choses que le datacore ne sait pas dire

Déjà annoncées au Tome I §7.6, elles sont rappelées ici parce qu'elles concernent le modèle de données :

Besoin	Datacore	Réalité
contrainte d'unicité	aucune notion	`@UniqueConstraint(name=..., columns=...)` à la main
relation obligatoire	`"nullable":"not"` déclaré…	…**jamais interpolé** par `BackendGenerator.php` ; la colonne de clé étrangère est NULLable dans tous les cas. `nullable=false` à ajouter à la main dans `@JoinColumn`
identité composite	aucune notion	`@Id @ManyToOne` à la main, pas de colonne `id`, `getId()` surchargée

Dans les trois cas, l'entité **ne doit plus être régénérée** : `core:g:crud` écrase l'annotation.

Chapitre 2 — Les entités

2.1 La chaîne d'héritage

```
stdClass
  └─ ModelSerializer         (dclass/devups/model/ModelSerializer.php)
       └─ Model              (dclass/devups/model/Model.php)
            │  use ModelTrait  (dclass/devups/model/ModelTrait.php)
            │  use DateTimeTrait
            └─ <Entité>       (src/<composant>/<Module>/Entity/)
```

Entre `Model` et l'entité, une **classe de base réutilisable** de `dclass/extends/` peut s'intercaler. Sur 3agReader :

Classe de `dclass/extends/`	Porte	Entités qui en héritent
`UserCore`	les colonnes communes d'un utilisateur (`username`, `firstname`, `lastname`, `email`, …)	`User`
`OrderCore`	les colonnes d'une commande (`totalamount`, `commission`, `discountprice`, `nettopay`, `kolacoin`, `status`, `due_date`, `$user`)	`Orders`
`ScanCore`	les colonnes communes d'une œuvre numérisée (`status`, `$category`, barrières de visibilité SQL)	`Comicbook`, `Chapter`
`ImageCore`	une image	`Dv_image`
`Item`		
`Dvups_config_item`		(déclarée, aucune table, aucun appelant — code mort)

Et deux traits :

Trait	Rôle
`RoleGuardTrait`	contrôle de rôle
`StatusTrait`	accès au statut
`CurrencyTrait`	devise

> **Règle.** `dclass/extends/` fait partie du framework. Un accesseur manquant s'ajoute **sur l'entité enfant**, dans `src/`. Appliqué sur 3agReader : `getUsername()` / `getFirstname()` sur `User` et non sur `UserCore` ; `getTotalamount()` / `getNettopay()` / `getNum_facture()` sur `Orders` et non sur `OrderCore`.

2.2 L'anatomie d'une entité

```php
<?php

class Task extends Model implements JsonSerializable
{
    /**
     * @Id @GeneratedValue @Column(type="integer")
     * @var int
     */
    protected $id;

    /**
     * @Column(name="name", type="string", length=255)
     * @var string
     */
    protected $name;

    /**
     * @Column(name="description", type="text", nullable=true)
     * @var string
     */
    protected $description;

    /**
     * @ManyToOne(targetEntity="\Project")
     * @JoinColumn(onDelete="cascade")
     * @var \Project
     */
    public $project;

    public function __construct($id = null)
    {
        $this->project = new Project();
    }

    public function getId()        { return $this->id; }
    public function getName()      { return $this->name; }

    public function jsonSerialize()
    {
        return ['id' => $this->id, 'name' => $this->name] + parent::jsonSerialize();
    }
}
```

Quatre conventions à respecter, toutes mesurées dans le code du framework :

1. Une colonne est `protected`, une relation est `public`. Ce n'est pas cosmétique : `Model-Trait::__get()` (`ModelTrait.php:44`) traite différemment une propriété déclarée et une propriété absente. Sur un attribut inconnu, il fait `trigger_error(...)` **puis `die`** — l'exécution s'arrête net, et l'écran entier disparaît.

> **L'exception du préfixe `_`.** Un attribut de la forme `_<relation>` n'est pas une erreur : c'est le **chargement paresseux** d'une relation. `$order->user->_wallet` hydrate le portefeuille à la demande et met le résultat en cache sur l'objet. C'est la seule forme d'attribut non déclaré que `__get()` accepte.

2. **Le constructeur instancie les relations.** `$this->project = new Project();` — sans cela, `$task->project->getName()` échoue sur `null`. Toutes les entités du socle le font.

3. **L'annotation doit toucher la classe.** `AnnotationDriver` ne lit que le docblock **immédiatement adjacent**. Un commentaire de prose en docblock entre l'annotation et `class Xxx` fait échouer toute lecture de métadonnées.

4. **`jsonSerialize()` ajoute, puis délègue.** `+ parent::jsonSerialize()` récupère la sérialisation du socle, qui remonte `$this->dvserialised` et les relations déclarées dans `$this->with`.

2.3 L'annotation de référence

Annotation	Portée	Rôle
`@Entity`	classe	déclare une entité Doctrine
`@Table(name="...")`	classe	le nom de table
`@Table(..., uniqueConstraints={@UniqueConstraint(name=..., columns={...})})`	classe	contrainte unique nommée
`@Id`	propriété	partie de l'identité
`@GeneratedValue`	propriété	auto-incrément
`@Column(name=, type=, length=, nullable=)`	propriété	une colonne
`@ManyToOne(targetEntity="\X")`	propriété	relation propriétaire
`@OneToMany(targetEntity="\X", mappedBy="y")`	propriété	relation inverse
`@ManyToMany(targetEntity="\X")`	propriété	relation n-n
`@JoinColumn(onDelete=, nullable=)`	propriété	la clé étrangère
`@var`	propriété	le type PHP ; lu par l'hydratation

2.4 Les propriétés `dv*`

`ModelTrait` déclare neuf propriétés de contrôle sur toute entité. Les connaître évite de les prendre pour des colonnes.

Propriété	Défaut	Rôle
`$dvfetched`	`false`	l'entité a déjà été hydratée depuis la base ; `hydrate()` ressort immédiatement si vrai
`$dvold`	`false`	l'état « avant » de l'entité, pour calculer le `SET` d'un `UPDATE`

Propriété	Défaut	Rôle
`$dvsoftdelete`	`false`	si vrai, `__delete()` écrit `deleted_at` au lieu de supprimer
`$dvinrelation`	`false`	l'entité est chargée comme relation d'une autre
`$dvtranslate`	`false`	**l'entité est traduite** — déclenche toute la mécanique du chapitre 7
`$dvid_lang`	`false`	la langue courante de l'instance
`$dvtranslated_columns`	`[]`	la liste des colonnes traduites
`$dvpersistence`	`[]`	les valeurs à écrire
`$dvserialised`	`[]`	les valeurs à sérialiser en JSON
`$with` (protégée)	`[]`	les relations à inclure dans `jsonSerialize()`

> **`$dvold` est le piège le plus coûteux du framework.** `Model::hydrate()` pose `$this->dvold = $this` — une **référence**, pas un clone. `entityKeyUpdate()` compare `$val != $this->dvold->{$key}` pour décider quoi écrire : comme les deux pointent le même objet, aucune différence n'est jamais détectée et l'`UPDATE` n'écrit rien. `Entity::find($id)`, lui, passe par `DBAL::__findOneRow()` qui pose un vrai `clone`. **Conséquence pratique : hydrater pour mettre à jour se fait toujours à partir d'un `find()`, jamais d'un `new X($id)`.** Détail complet au Tome I §6.7.

Chapitre 3 — Les trois classes de la couche d'accès

3.1 L'architecture

```
Model  (dclass/devups/model/Model.php)
  └─ use ModelTrait
       → les points d'entrée STATIQUES : Entity::find(), ::where(), ::create(), …
       → chaque point d'entrée fabrique un `new QueryBuilder($entity)` et lui délègue

QueryBuilder  (dclass/devups/Tchutte/QueryBuilder.php)
  extends DBAL
       → construit le SQL (SELECT / INSERT / UPDATE / WHERE / JOIN / ORDER BY…)
       → API fluide : chaque méthode `return $this`

DBAL  (dclass/devups/Tchutte/DBAL.php)
  extends Database  (config/Database.php)
       → exécute le SQL et hydrate les entités
       → expose la connexion PDO partagée : DBAL::pdo()
```

Le nom « Tchutte » vient du surnom de l'auteur du socle, pas d'un produit externe.

3.2 Doctrine, et ce qu'il ne fait pas

`Doctrine\ORM\EntityManager`, accessible par `DBAL::getEntityManager()`, est une **quatrième brique séparée**. Elle sert exactement à deux choses :

1. lire les métadonnées d'annotation — c'est ainsi que le framework sait quelle table, quelles colonnes et quelles associations porte une entité ;
2. `schema:update`, les migrations.

Aucune requête métier ne passe par Doctrine. Toute lecture, écriture et agrégation passe par `DBAL` / `QueryBuilder`. Ne pas aller chercher une connexion à partager dans `getEntityManager()` : c'est une connexion différente, volontairement.

3.3 La connexion partagée

```
$pdo = Model::pdo();   // délègue à DBAL::pdo()
```

Retourne **le même objet PDO** que celui utilisé par `find()`, `__insert()`, `__update()` et tout le query builder — vérifié par `spl_object_id()` depuis plusieurs points d'entrée.

À utiliser pour toute requête SQL qui ne passe pas par le query builder, par choix ou par nécessité. **Jamais** `new PDO(...)` : cela ouvrirait une connexion séparée, invisible à toute transaction ouverte par `Model::beginTransaction()`.

> **Historique utile.** Avant une correction mesurée sur BGFIBank Queue (F-01/F-03), `DBAL::__construct()` mettait en cache **une connexion différente par nom d'entité**. Un `Ticket::find()` et un `Print_job::find()` utilisaient deux connexions MySQL distinctes, et une transaction ouverte sur l'une n'englobait pas l'autre. C'est désormais une connexion unique par processus.

Chapitre 4 — Lire

4.1 Les points d'entrée statiques

Toute entité hérite de `ModelTrait`, qui expose ces méthodes statiques directement sur le nom de la classe.

Méthode	Rôle	Retour sur zéro résultat
`Entity::find($id, $id_lang = null)`	lecture par id (ou tableau d'ids, ou tableau de conditions)	**entité vide**, jamais `null` ; `[]` si `$id` est un tableau numérique
`Entity::findOrNull($id, $id_lang = null)`	idem, nullable	`null`
`Entity::findrow($id, $id_lang = null)`	forme historique de `find()`	entité vide
`Entity::where($col, $op, $val, $id_lang = null)`	fabrique un `QueryBuilder` déjà filtré	— (chaîner)
`Entity::whereId($id, $id_lang = null)`	`where('this.id', '=', $id)`, ou `whereIn` si tableau	—
`Entity::whereIn($col, $collection)`	—	—
`Entity::where_str($col, $link, $id_lang)`	contrainte SQL brute	—
`Entity::first($id_lang = null)`	le premier enregistrement, sans filtre	entité vide
`Entity::firstOrNull($id_lang = null)`	idem, nullable	`null`
`Entity::last($id_lang = null)` / `::las-trow()`	le dernier	entité vide
`Entity::index($index, $id_lang)`	le n-ième	
`Entity::firstOrCreate($constraint, $data, $id_lang)`	lit, ou crée si absent	crée toujours
`Entity::firstOrNew($constraint, $data, $id_lang)`	lit, ou instancie sans insérer	—
`Entity::count($col = null, $val = null)`	`COUNT(*)`, filtré ou non	`int`
`Entity::sum($cols)` / `::avg()` / `::max()` / `::min()` / `::selectSum($col)`	agrégats	
`Entity::all($sort='id', $order='asc', $id_lang)`	tout	`[]`
`Entity::allrows(...)`	tout, en lignes brutes	`[]`
`Entity::trashed(...)`	les lignes en suppression douce	`[]`
`Entity::getattribut($attribut, $id)`	une seule colonne, par id	

Méthode	Rôle	Retour sur zéro résultat
`Entity::getbyattribut($attribut, $value, $id_lang)`	une entité, par n'importe quelle colonne	
`Entity::select($columns='*', $id_lang, $defaultjoin=true)`	un `QueryBuilder` avec projection	
`Entity::initQb($defaultjoin = true)`	**un `QueryBuilder` vierge**, prêt à tout chaîner	
`Entity::initBulk()`	insertion en masse	
`Entity::with($entity, $attr = [])`	chargement de relation	
`Entity::join(...)` / `::leftJoinOn(...)`	jointures	
`Entity::orderBy($col, $sort)`	tri	
`Entity::distinct($cols)`		
`Entity::lazyloading($order = "")`	listing paginé de l'API (voir Tome III)	
`Entity::truncate()`	vide la table	
`Entity::dclone($id, $update)`	duplique une ligne	

`Entity::where(...)` est le point d'entrée du quotidien : il équivaut à `Entity::initQb()->where(...)`.

`$defaultjoin = false` (sur `initQb()`, ou en troisième argument du constructeur de `QueryBuilder`) désactive les `LEFT JOIN` automatiques des relations `manyToOne` — utile quand la requête n'a besoin d'aucune relation et que les jointures ne seraient que du bruit.

4.2 La clause `WHERE`

Toutes ces méthodes vivent sur `QueryBuilder` et retournent `$this`.

Comparaisons

```
->where('this.status', '=', 'active')          // opérateur explicite
->where('this.id', $id)                         // deux arguments = égalité
->where(['status' => 'active', 'type' => 'X'])  // tableau = plusieurs égalités en AND
->andwhere($col, $op, $val)                     // alias explicite en AND
->orWhere($col, $op, $val)                      // lié en OR
->whereId($id)                                  // this.id = $id, ou IN si tableau
```

`$column` accepte aussi **directement une entité** : `->where($uneInstanceDeProject)` génère `project.id = :project_id` (ou `IS NULL` si l'entité n'a pas d'id) — pratique pour filtrer par relation sans écrire `->where('this.project_id', '=', $project->getId())`.

Le préfixe `this.` désigne l'entité de base. Sans préfixe, la colonne est ambiguë dès qu'une jointure est en jeu — voir §4.8.

Les opérateurs reconnus

La propriété privée `$operators` (`QueryBuilder.php:51`) contient exactement :

```
=   <   >   <=   >=   <>   !=   <=>
like   'like binary'   'not like'   'not ilike'   ilike   between   is
&   |   ^   <<   >>
rlike   regexp   'not regexp'
~   ~*   !~   !~*   ~~*   !~~*
'similar to'   'not similar to'
```

> **Un opérateur absent de cette liste ne lève pas d'erreur.** Il tombe dans une branche qui **ignore silencieusement** `$value` et lie l'opérateur lui-même comme valeur (`$this->parameters[$attrib] = $operator`). La requête s'exécute et renvoie un résultat faux. C'est un défaut du framework, pas une forme à reproduire : rester sur la liste ci-dessus, ou passer par une méthode dédiée.

Recherche « contient »

```
->whereLike('label', $q)          // LIKE '%q%', en AND
->whereLike('label', $q, 'l')     // LIKE 'q%'   (préfixe)
->whereLike('label', $q, 'r')     // LIKE '%q'   (suffixe)
->orWhereLike('city', $q)         // en OR
->where('col')->like($q)          // forme chaînée en deux appels
->where('col')->_like($q)         // variantes
->where('col')->like_($q)
```

Ces méthodes sont **paramétrées** depuis une correction mesurée (F-05b) : elles interpolaient la valeur directement dans le SQL, c'était une injection. Un compteur interne (`$likeParamSeq`) évite les collisions de noms de paramètres dans une même requête.

Groupe de conditions en OR

Le filtrage dynamique de l'API ne combine qu'en ET. Pour une recherche multi-champs en OU, `whereGroup()` prend une fermeture qui construit ses conditions sur le même builder, puis les regroupe entre parenthèses :

```
Agency::where('this.status', '=', 'active')
    ->whereGroup(function ($qb) use ($q) {
        $qb->whereLike('label', $q)
           ->orWhereLike('city', $q)
           ->orWhereLike('address', $q);
    })
    ->orderBy('this.label')
    ->get();

// WHERE ... AND status = :x AND (label LIKE :a OR city LIKE :b OR address LIKE :c)
```

Un second argument `$outerLink` (`'AND'` par défaut) rattache le groupe entier en OU au reste.

À ne pas confondre avec `whereRaw($constraint, $link)` et `where_str($constraint, $link)` — deux méthodes au **corps strictement identique** — qui prennent une chaîne SQL brute, jamais paramétrée. Dernier recours, jamais avec une valeur venant d'un utilisateur.

Listes

```
->whereIn('this.id', [1, 2, 3])              // IN (1,2,3)
->whereIn('this.status', ['a','b'], 'NOT')   // NOT IN
->where('col')->in([1,2,3])                  // forme chaînée
->where('col')->notIn($values)
->whereInReverse($value, ['col1','col2'])    // $value IN (col1, col2) — rare
```

> **Non corrigé.** `whereIn()` quote ses chaînes via `qb_sanitize()` (`dvups_functions.php`), qui fait littéralement `return "'$value'";` — **sans échapper**. Un tableau de valeurs contrôlées par un utilisateur y reste injectable. Caster ou valider avant (`(int)`, liste blanche), ou passer par `whereGroup()` + `where('col','=',$v)` répété.

Null, dates, intervalles

```
->whereNull('col')      ->whereNotNull('col')
->whereBetween('col', [$min, $max])     // interpolation directe — mêmes réserves
->where('col')->between($v1, $v2)
->whereDay('col', $v)   ->whereMonth('col', $v)   ->whereYear('col', $v)
->whereDate('col', $v)  ->whereHour('col', $v)
```

4.3 Tri, regroupement, limite

```
->orderBy('this.label')              // ASC implicite
->orderBy('this.label', 'DESC')
->orderBy('col1')->orderBy('col2')   // cumulables, la virgule est ajoutée
->rand()                              // ORDER BY RAND()
->groupBy('this.project_id')
->limit(10)                           // LIMIT 10
->limit(0, 10)                        // LIMIT 0, 10
->limit(-5)                           // les 5 DERNIERS (compte le total en interne)
->distinct('col')
->index($columnOrder, $alias)         // ajoute un index de position
```

`orderBy('project.position')` — une colonne d'une relation jointe — fonctionne directement **si** la relation est jointe par défaut (§4.4).

4.4 Les relations

Chaque `@ManyToOne(targetEntity="\X")` de l'entité de base est **automatiquement jointe en `LEFT JOIN`** dès qu'un `QueryBuilder` est construit avec `$defaultjoin = true`, c'est-à-dire par défaut. L'alias de jointure est le nom de la table cible en minuscules :

```
LEFT JOIN `project` ON `project`.id = task.project_id
```

> **Ce `LEFT JOIN` ne charge que l'`id`.** Accéder à `$task->project->name` sans rien de plus renvoie **une chaîne vide** — pas une erreur, pas de latence perceptible, rien de visible.

```
// SANS with() : $row->project existe (objet non nul), toutes ses colonnes
// hors id sont vides.
$rows = Task::where('this.done', '=', 0)->get();
echo $rows[0]->project->name;   // "" — pas d'erreur, juste vide

// AVEC with() : les colonnes demandées sont réellement hydratées
$rows = Task::where('this.done', '=', 0)
    ->with('project', ['name', 'description'])
    ->get();
echo $rows[0]->project->name;   // "Refonte du catalogue"
```

Signature : `with(string $entityName, array $columns = [])`. `$entityName` est le nom de la **propriété de relation** telle que déclarée dans l'entité (`public $project;` → `'project'`), pas le nom de la classe cible si les deux diffèrent.

Méthode	Effet
`with($relation, $columns)`	hydrate réellement les colonnes de la relation
`without(...$relations)`	exclut des relations du `LEFT JOIN` par défaut
`initQb(false)`	désactive **tous** les `LEFT JOIN` par défaut
`join($classname, $on, $id_lang)`	jointure explicite
`leftjoin($classname, $on, $constraint)` / `leftJoinOn($classname, $alias, $constraint)`	
`innerjoin(...)` / `innerJoinOn(...)`	
`join_str($classname, $on)`	jointure en SQL brut
`leftjoinrecto($classname, $on, $alias)`	

`with()` est aussi pilotable **depuis la requête HTTP**, sans toucher au code : l'en-tête `Columns` portant un JSON `{"project":["name","description"]}` est converti en appels `with()` par `Lazyloading::filter()`. Pratique pour explorer une API.

4.5 Les méthodes terminales, et le piège `null` contre entité vide

C'est le piège le plus coûteux de cette couche. Deux familles de méthodes terminales, avec un comportement différent sur « aucun résultat » :

Méthode	Zéro résultat
`->get($column = "*", $id_lang = null)`	`[]`
`->getInstance($column = "*")` / `->first($id_lang)`	**objet entité VIDE** (`new $this->objectName`), jamais `null`
`->firstOrNull($id_lang)`	`null`
`->lastOrNull($id_lang)` / `->getLastOrNull($id_lang)`	`null`
`Entity::find($id)` (id scalaire)	objet entité VIDE

Méthode	Zéro résultat
`Entity::findOrNull($id)`	`null`
`->getRows($column, $callback)`	tableau de lignes brutes, sans hydratation
`->row()` / `->__getOneRow()`	une ligne brute
`->getValue()` / `->getValues()`	valeur(s) scalaire(s)
`->count($col, $as)` / `->sum()` / `->avg()` / `->max()` / `->min()`	scalaire
`->trashed($column, $id_lang)`	les lignes en suppression douce
`->cursor($callback, $return_data, $cache)`	parcours ligne à ligne
`->getIndex($index, $id_lang)` / `->getInstanceAtIndex($index)`	la n-ième
`->firstOr($callback)` / `->firstOrCreate()` / `->firstOrNew()`	

Conséquence directe : après un `find()`, `getInstance()` ou `first()` **sans** le suffixe `OrNull`, ne jamais tester `if ($entity)` seul — un objet PHP est toujours vrai.

```
// FAUX — ne détecte jamais une entité vide
$existing = Agency::where('this.code', '=', $code)->getInstance();
if ($existing) { /* toujours vrai */ }

// CORRECT
$existing = Agency::where('this.code', '=', $code)->firstOrNull();
if ($existing && $existing->getId()) { /* vraiment trouvé */ }
```

Le motif `if ($x && $x->getId())` est systématique dans le code du socle. C'est la forme à reprendre.

4.6 Prévisualiser une requête sans l'exécuter

```
QueryBuilder::$debug = true;

$result = Agency::where('this.status', '=', 'active')->get();
// $result n'est PAS le résultat : c'est
// [
//   'query'      => "... valeurs en ligne, lisible ...",
//   'sql'        => "... avec les :placeholders ...",
//   'parameters' => ['this_status' => 'active', ...]
// ]

QueryBuilder::$debug = false;   // à ne pas oublier
```

Utile pour vérifier une jointure ou un `whereGroup()` avant exécution, et pour **prouver** qu'une valeur est bien liée et non interpolée : une valeur contenant une apostrophe reste dans `parameters`, jamais dans le texte SQL.

`$debug` est une propriété **statique** : elle affecte tout `QueryBuilder` construit tant qu'elle est vraie, pas seulement l'instance courante. La remettre à `false` immédiatement.

`->getSqlQuery($column)` donne le SQL d'une requête sans passer par le mode debug global.

4.7 Exemples prêts à reprendre

Lecture par id avec « non trouvé » explicite

```
$agency = Agency::findOrNull($id);
if (!$agency) {
    return ['success' => false, 'detail' => 'agence introuvable'];
}
```

Liste filtrée et triée

```
$active = Agency::where('this.status', '=', 'active')
    ->orderBy('this.label')
    ->get();
```

Jointure avec chargement réel des colonnes

```
$offered = Agency_service::where('this.agency_id', '=', $id)
    ->where('this.active', '=', 1)
    ->with('service', ['code', 'label', 'icon'])
    ->orderBy('service.position')
    ->get();

foreach ($offered as $row) {
    echo $row->service->code . ' : ' . $row->service->label;   // hydraté
}
```

Recherche multi-champs en OU

```
$qb = Agency::where('this.status', '=', 'active');

if (($q = trim($q)) !== '') {
    $qb->whereGroup(function ($sub) use ($q) {
        $sub->whereLike('label', $q)
            ->orWhereLike('city', $q)
            ->orWhereLike('address', $q);
    });
}

$agencies = $qb->orderBy('this.label')->get();

// Mise en forme explicite : garder le contrat de réponse figé, jamais un
// jsonSerialize() de l'entité qui exposerait toutes ses colonnes.
return ['success' => true, 'agencies' => array_map(fn($a) => [
    'id' => $a->getId(), 'code' => $a->code, 'label' => $a->label,
], $agencies)];
```

Vérifier l'existence avant d'écrire

```
// DBAL::executeDbal() n'est pas rattrapable par un try/catch de contrôleur :
// toujours vérifier AVANT d'écrire, pas après.
$existing = Agency::where('this.code', '=', $code)->firstOrNull();
if ($existing && $existing->getId()) {
    return ['success' => false, 'detail' => "code déjà utilisé"];
}
```

Compter

```
$total = Ticket::where('this.agency_id', '=', $agencyId)
    ->where('this.status', '=', 'waiting')
    ->count();
```

4.8 Qualifier les colonnes : un défaut à connaître

Quand une entité est **traduite** (`$dvtranslate = true`), la jointure de traduction ramène la table `dvups_lang` sous l'alias `l`. Cette table a sa propre colonne `name`.

Conséquence : sur une entité traduite qui a elle aussi une colonne `name`, un `WHERE name = ...` non qualifié devient

```
SQLSTATE[23000] ... Column 'name' in where clause is ambiguous
```

Sur 3agReader, cela rendait `Tree::where("name", $x)` — et tout ce qui s'appuyait dessus — **inutilisable**. Sept occurrences réelles ont été trouvées et corrigées en qualifiant (`"this.name"`), mais **la racine dans `QueryBuilder::initSelect()` n'est pas corrigée** : tout nouvel appelant qui oublie de qualifier reproduit l'erreur.

> **Règle de travail** : sur une entité traduite, **toujours** qualifier (`this.<colonne>`). Sur les autres aussi, par habitude : cela ne coûte rien et prévient le jour où l'entité devient traduite.

Chapitre 5 — Écrire

5.1 Les formes

```
// création
$id  = Entity::create(['col1' => 'val', 'col2' => 42]);        // un id
$ids = Entity::create(['a' => 1], ['a' => 2]);                   // plusieurs lignes, un ta-
bleau d'ids
$e   = Entity::createInstance([...]);                            // l'instance, pas l'id
$e   = Entity::createFrom($data, $table_targeted);

// mise à jour
$entity = Entity::find($id);
$entity->col1 = 'nouvelle valeur';   // __set magique, même sur une propriété protected
$entity->__update();                  // UPDATE de toutes les colonnes de l'objet
$entity->__update(['col1' => 'x']);   // UPDATE ciblé
$entity->__save();                    // insert ou update selon la présence d'un id

// bas niveau
$entity->__insert();
$entity->__delete();                  // DELETE, ou suppression douce si dvsoftdelete
$entity->__dclone($update);           // duplique la ligne

// statiques
Entity::update($arrayvalues, $primarykey, $where, $classname);
Entity::delete($id, $force = false);
```

Il n'y a pas de `setXxx()` fluide généré. L'affectation directe (`$entity->col = valeur`) est le mécanisme, via le `__set` magique de `ModelTrait`. Un `setXxx()` écrit à la main a un rôle différent : il est appelé par le traitement de formulaire (voir §5.4).

5.2 Deux pièges d'écriture

1. **`Entity::create()` attend le nom de colonne physique pour une relation.**

```
Orders_item::create(['orders_id' => 3]);   // correct
Orders_item::create(['orders' => 3]);       // échoue : "Unknown column 'orders'"
```

Le message d'erreur ne nomme pas la vraie cause. La forme `'relation.id' => 3` fonctionne dans la **charge utile d'API** (voir Tome III), pas dans `create()`.

2. **`DBAL::executeDbal()` n'est pas rattrapable depuis un contrôleur.** Une violation de contrainte ne remonte pas comme une exception qu'un `try/catch` de contrôleur peut traiter proprement. **Vérifier avant d'écrire**, jamais après.

5.3 Les transactions

```
Model::beginTransaction();   // niveau 0 : vraie transaction PDO. niveau > 0 : SAVEPOINT
try {
    $ticket->__insert();
    $printJob->__insert();
    Model::commit();          // niveau 0 : COMMIT réel. niveau > 0 : rien
} catch (\Throwable $e) {
    Model::rollBack();        // niveau 0 : ROLLBACK réel.
                              // niveau > 0 : ROLLBACK TO SAVEPOINT (ce niveau seul)
    throw $e;
}
```

La forme recommandée protège contre un `rollBack()` oublié sur un retour anticipé :

```
Model::transaction(function () use ($ticket, $printJob) {
    $ticket->__insert();
    $printJob->__insert();
    return $ticket->getId();   // un retour anticipé ici valide quand même :
                               // PHP rend la main au bloc, qui enchaîne sur commit()
});
```

L'imbrication est transparente. Deux méthodes qui appellent chacune `Model::transaction()` peuvent s'appeler l'une l'autre sans jamais lever le « There is already an active transaction » de PDO : seul le niveau 0 touche la connexion, les niveaux internes posent et libèrent un `SAVEPOINT`.

> **Contrepartie de la connexion unique** : une transaction laissée ouverte empoisonne **tout** ce qui suit dans la même requête HTTP. Toujours un `try/catch`, ou mieux `Model::transaction()` ; jamais un `beginTransaction()` isolé sans son `commit()` / `rollBack()` garanti.

Écriture transactionnelle multi-tables

```
Model::transaction(function () use ($agencyId, $serviceId) {
    $ticketId = Ticket::create([...]);
    Ticket_event::create(['ticket_id' => $ticketId, ...]);
    (new Print_jobFrontController())->enqueue(...);
    return $ticketId;
});
```

5.4 Les accesseurs et le traitement de formulaire

Un `setXxx()` écrit à la main n'est pas un accesseur fluide : c'est un **point de validation** appelé par `Controller::formWithPost()`. Son contrat de retour est particulier :

```
$error = call_user_func([$object, $setter], ...);
if ($error) $this->error[$key] = $error;
```

Il n'y a **aucun** `try/catch` autour. Un setter de validation doit donc **retourner une chaîne d'erreur**, pas lever une exception :

```
public function setBranches($branches)
{
    $segments = array_map('trim', explode(',', $branches));
    foreach ($segments as $s) {
        if ($s === '') {
            return "Un nom de palier ne peut pas être vide "
                 . "(virgule en double ou en bordure).";
        }
    }
    $this->branches = implode(',', $segments);
    return null;   // pas d'erreur
}
```

Le socle le fait déjà ainsi : `Dvups_admin::setName()` retourne `"name empty"`. Conséquence agréable : exposer un nouveau champ validé ne demande **aucune** modification du contrôleur.

5.5 Sérialisation

`ModelSerializer::jsonSerialize()` remonte `$this->dvserialised` plus, pour chaque clé de `$this->with`, la sérialisation de la relation correspondante.

Une entité surcharge en ajoutant, puis en déléguant :

```
public function jsonSerialize()
{
    return [
        'id'   => $this->id,
        'name' => $this->name,
    ] + parent::jsonSerialize();
}
```

> **Pour une réponse d'API, préférer une mise en forme explicite à `jsonSerialize()`.** Sérialiser l'entité expose toutes ses colonnes, y compris celles qui n'ont jamais été documentées comme publiques, et fige involontairement le contrat de réponse sur la structure de la table. Le motif de `AgencyFrontController::directory()` (§4.7) construit la réponse champ par champ.

Chapitre 6 — Les classes de base d'entité du socle

6.1 `ScanCore` — les œuvres numérisées

`dclass/extends/ScanCore.php` est la base de `Comicbook` et `Chapter` sur 3agReader. Elle porte :

```
/** @Column(name="status", type="integer") */
protected $status = 0;

/**
 * @ManyToOne(targetEntity="\Tree_item")
 * @JoinColumn(onDelete="set null")
 */
public $category;
```

et surtout les **barrières de visibilité**, écrites en SQL brut : `seriesVisibleSql()`, `publishedSql()`, `chapterSeriesDateSql()`. Elles comparent la colonne `status` à des **entiers littéraux** (`status = 1`, `status = 0`), jamais par une jointure vers une table de statuts.

> **C'est une bonne séparation, pas un défaut.** Faire dépendre une barrière de visibilité d'une table éditable par un administrateur — où l'on peut ajouter, renommer ou supprimer une clé — serait fragile. La table de statuts (Tome IV) décrit *ce que les entiers veulent dire pour un humain* ; elle ne décide jamais *ce qui est visible*.

6.2 `OrderCore` — les commandes

```
protected $totalamount    = 0;   // @Column(type="float")
protected $commission     = 0;   // @Column(type="float")
protected $discountprice  = 0;   // @Column(type="float")
protected $nettopay       = 0;   // @Column(type="float")
protected $kolacoin       = 0;   // @Column(type="float")
protected $status;                // @Column(type="integer", nullable=true)
protected $due_date;              // @Column(type="date", nullable=true)

/**
 * @ManyToOne(targetEntity="\User")
 * @JoinColumn(onDelete="Cascade")
 */
public $user;
```

> **Dette à connaître avant de s'appuyer dessus.** Les cinq colonnes d'argent sont déclarées `type="float"`. Un flottant binaire ne représente pas exactement un montant décimal : une répartition en pourcentages et une conversion de devise ne sont pas reproductibles à l'octet près, et les écarts s'accumulent. La discipline attendue sur des montants est `DECIMAL(n,2)`, ou des entiers de centièmes. Ces colonnes vivent dans `dclass/extends/`, donc leur migration a un rayon d'effet qui dépasse un projet : c'est un arbitrage de l'auteur du socle, pas une correction à faire au passage. Documenté ici pour que personne ne construise un calcul d'argent dessus en croyant qu'il est exact.

6.3 `UserCore` — les utilisateurs

Porte les colonnes communes d'un utilisateur. Deux points mesurés sur 3agReader :

- `username` est **l'adresse de courriel** dans le flux d'inscription (un `setUsername()` historique valide la valeur avec `FILTER_VALIDATE_EMAIL`), mais pas toujours : le compte par défaut du socle a `username = "3agEdition"`. Une liste d'administration qui affiche `User.username` affiche donc le plus souvent une adresse de courriel. À savoir avant d'exposer la colonne.
- Aucun accesseur n'était défini sur `username` / `firstname` — ni sur `UserCore`, ni sur `User`. Trois écrans du back-office plantaient sur `class User does not have a method "getUsername"`. Les accesseurs ont été ajoutés **sur `User`**, dans `src/`.

Chapitre 7 — La traduction (`dvtranslate`)

7.1 Le modèle

Une entité traduite déclare :

```
public $dvtranslate = true;
public $dvtranslated_columns = ["label"];
```

Son schéma se dédouble :

Table	Contenu
`<entité>`	les colonnes non traduites, dont la **clé machine**
`<entité>_lang`	une ligne par langue, avec les colonnes traduites et `lang_id`
`dvups_lang`	les langues (`id`, `iso_code`, `active`, `name`)

Sur 3agReader, les entités réellement traduites sont :

Entité	Table de langue
`Tree`	`tree_lang`
`Tree_item`	`tree_item_lang`
`Category`	`category_lang`
`Status`	`status_lang`
`Cmstext`	`cmstext_lang`
`Notificationtype`	`notificationtype_lang`
`Reportingmodel`	`reportingmodel_lang`
`Local_content`	`local_content_lang`

Plus `Dvups_config_item` (`dclass/extends/`), qui déclare `dvtranslate = true` mais n'a **aucune table réelle** et **aucun appelant** : code mort.

7.2 La bonne séparation

Le schéma sépare déjà correctement **la clé machine** et **le libellé traduit** :

Entité	Clé machine (non traduite)	Libellé (traduit)
`Tree`	`tree.name`	`tree_lang.label`
`Tree_item`	`tree_item.slug`	`tree_item_lang.name`

C'est la forme à reproduire : **le code se réfère à la clé machine, jamais au libellé.** Un libellé change quand un traducteur le change ; une clé machine ne doit jamais changer.

7.3 Forcer une langue

Appel	Effet
`->setLang($id_lang)`	fixe la langue du builder
`Entity::find($id, $id_lang)`	
`->firstOrNull($id_lang)` / `->first($id_lang)`	
`->get($col, $id_lang)`	**attention**, voir §7.6
en-tête HTTP `Dlang`	positionne `Request::$uri_header_param['dlang']`
paramètre `?dlang=fr`	lu par `FrontController::detailCore()`

Sans langue forcée, les colonnes traduites reviennent **concaténées sur toutes les langues** par un `GROUP_CONCAT`, et le consommateur les sépare — c'est ce qui produit l'objet bilingue `{fr: ..., en: ...}` que l'application mobile de 3agReader reçoit.

7.4 Le défaut D15 — les lignes invisibles

C'est le défaut le plus sérieux de cette couche, et il touche toute entité traduite, pas une en particulier.

Le mécanisme, à l'endroit exact

`QueryBuilder::initSelect()` (`QueryBuilder.php:1582-1600`) — appelé par `getInstance()`, `get()`, `getValue()`, `getValues()`, `row()`, `trashed()`, donc par `find()`, `findOrNull()`, `first()`, `firstOrNull()`, `where(...)->get()` : **toute lecture ORM** :

```
if ($this->object->dvtranslate && !$aggregation) {
    $thislang = $this->table . "_lang";
    if ($this->id_lang) {
        // (a) langue forcée — échappe au défaut
        $this->_join .= " left join $thislang on ($thislang.{$this->table}_id
                        = `{$this->table}`.id
                        and $thislang.lang_id = {$this->id_lang}) ";
    } else {
        // (b) branche par défaut — LE DÉFAUT
        $this->_join .= " left join $thislang on ($thislang.{$this->table}_id
                        = `{$this->table}`.id) ";
        $this->_join .= " join dvups_lang l on ($thislang.lang_id = l.id) ";
        //                ^^^^ INNER JOIN, pas LEFT
        $this->_group_by .= ("{$this->table}.id");
    }
}
```

La branche par défaut chaîne un `LEFT JOIN <table>_lang` **sans filtre** — pour remonter toutes les langues et les concaténer — avec un `JOIN dvups_lang l` **(INNER)** sur `<table>_lang.lang_id = l.id`.

Si la ligne de base n'a **aucune** ligne `<table>_lang`, le `LEFT JOIN` produit des colonnes `NULL`, et le `JOIN` suivant sur `NULL = l.id` **élimine la ligne entière**, pour tout le reste de la requête — y compris un `WHERE this.id = X`.

La ligne existe en base, n'est pas perdue, et devient invisible à l'ORM. Silencieusement.

Les deux SQL, mesurés

Sur `Tree::where("this.name", "menu")`, id réel 5, zéro ligne `tree_lang` :

Sans langue forcée (`->firstOrNull()`) :

```
SELECT ... FROM `tree`
  left join tree_lang on (tree_lang.tree_id = `tree`.id)
  join dvups_lang l on (tree_lang.lang_id = l.id)
WHERE 1 AND `tree`.name = menu GROUP BY tree.id
```

Résultat : `NULL`. La ligne est invisible.

Avec langue forcée (`->firstOrNull($langId)`) :

```
SELECT ... FROM `tree`
  left join tree_lang on (tree_lang.tree_id = `tree`.id and tree_lang.lang_id = 1)
WHERE 1 AND `tree`.name = menu
```

Résultat : l'objet `id=5, name='menu'`. **Visible.** Il n'y a pas de `JOIN dvups_lang` du tout dans cette branche : le filtre de langue est dans la clause `ON` du `LEFT JOIN`, rien à éliminer.

Ce que ça coûte, chiffré

Sur 3agReader, au moment de la mesure :

Entité	Lignes	Invisibles
`Tree`	16	**9**
`Tree_item`	95	0
`Category`	5	0
`Status`	7	0
`Cmstext`	9	0
`Notificationtype`	21	0
`Reportingmodel`	5	0
`Local_content`	87	0

Le défaut est **réel et latent partout**, mais seul `Tree` a aujourd'hui des données qui le déclenchent. Les neuf arbres invisibles redeviennent tous visibles en forçant la langue ; la colonne traduite revient alors `NULL`, jamais une erreur et jamais une clé absente — il n'y a simplement rien à lire.

Le contournement, et pourquoi la racine n'est pas corrigée

Contournement : forcer une langue. C'est un contournement existant du cadre applicatif, pas une réparation.

Pourquoi la racine n'est pas corrigée : `initSelect()` est partagé par toute entité traduite, dans tout module, dans tout projet basé sur ce socle. Remplacer le `JOIN` par un `LEFT JOIN` changerait le résultat de chaque lecture traduite du framework. C'est un arbitrage de l'auteur du socle, pas une correction à glisser dans un chantier applicatif.

Manifestation côté écriture. Le même défaut se voit aussi à l'enregistrement : mettre à jour une entité traduite qui n'a aucune ligne `_lang` produit une erreur SQL malformée (`WHERE lang_id = '1' AND tree_id = '5'`) dans la synchronisation de traduction de `DBAL`. Un garde a été posé sur 3agReader dans `Model::__update()` : avant une mise à jour complète d'une entité traduite, il refuse explicitement avec un motif clair si la fiche n'a pas de traduction — plutôt que de laisser remonter une erreur SQL illisible pour un administrateur.

L'outil de diagnostic

`DvtranslateInspector` (`dclass/devups/model/DvtranslateInspector.php`, ajouté par 3agReader, **lecture seule**) :

Méthode	Rôle
`invisibleRows($entityClass)`	compte et liste les lignes invisibles d'une entité
`isInvisible($entityClass, $id)`	cet id existe-t-il en base tout en étant invisible à l'ORM ?
`diagnose($entityClass, $id)`	distingue `no_translation`, `orphan_lang`, `visible`, `not_found`, `not_translated`, `schema_missing`

Sur une entité non traduite ou une classe inconnue, il refuse explicitement : **aucune requête sur une table `_lang` n'est jamais construite à l'aveugle**.

Le cas `orphan_lang` est structurellement impossible

Une ligne de traduction pointant vers une langue supprimée ne peut pas exister : les huit tables `*_lang` réelles portent chacune `FOREIGN KEY (lang_id) REFERENCES dvups_lang(id) ON DELETE CASCADE` (vérifié une par une dans `information_schema`). Une écriture orpheline est rejetée, et supprimer une langue supprime en cascade ses traductions. Le cas reste traité par l'outil pour rester correct si le schéma changeait, mais il n'est pas reproductible aujourd'hui.

7.5 La bonne pratique, quand on doit lire une colonne non traduite

Si le besoin est de lire une colonne **non traduite** d'une entité traduite — une clé machine, un schéma, un drapeau — et que la ligne peut ne pas avoir de traduction, passer par une requête préparée brute :

```
$stmt = \Model::pdo()->prepare("SELECT branches FROM tree WHERE name = ?");
$stmt->execute([$treeName]);
$branches = $stmt->fetchColumn();
```

C'est ce que fait `Tree::buildCacheData()` sur 3agReader, précisément parce que l'arbre dont il a besoin est l'un des neuf sans traduction, et que l'ORM l'aurait manqué en silence.

7.6 Un second défaut, dans la même famille

`QueryBuilder::get($colonne, $id_lang)` appelle `initSelect()` **avant** de poser `$this->id_lang` quand celui-ci n'était pas déjà fixé. **Passer `$id_lang` en second argument de `get()` est donc sans effet** sur le cadrage de langue : le libellé revient concaténé sur toutes les langues.

La forme correcte est d'appeler `setLang()` **avant** `get()` :

```
$rows = Status::where('this.entity', '=', 'comicbook')
    ->setLang($id_lang)
    ->get();           // sans argument
```

C'est le motif déjà en usage dans le dépôt (`Tree_item::getmainmenu()`).

Chapitre 8 — Pièges et vulnérabilités de la couche de données

Récapitulatif, avec l'état réel de chacun.

#	Objet	État	Détail
1	`whereLike()`, `like()`, `_like()`, `like_()`	**corrigé**	interpolaient la valeur dans le SQL — injection. Paramétrées depuis, même signature.
2	`whereIn()` sur des chaînes	**non corrigé**	`qb_sanitize()` entoure de quotes **sans échapper**. Valider ou caster avant.
3	`whereBetween()`	**non corrigé**	même interpolation directe que l'ancien `whereLike()`.
4	`where($col, $opérateur_inconnu, $val)`	comportement du framework	un opérateur hors de la liste `$operators` **ignore `$val`** et lie l'opérateur comme valeur. Résultat faux, sans erreur.
5	relations `manyToOne`	documenté	hydratation paresseuse : `with()` est requis au-delà de l'`id`.
6	`getInstance()`, `first()`, `find($id)` sur zéro résultat	documenté	renvoient une entité **vide**, jamais `null`. Toujours `if ($x && $x->getId())`.
7	`whereRaw()` / `where_str()`	duplication	deux méthodes au corps identique. Laissées telles quelles : les renommer casserait des appelants pour un gain cosmétique.
8	connexion PDO	**corrigé**	une seule connexion par processus (avant : une par entité).
9	`$dvold` par référence	**contourné dans le code généré**	`Model::hydrate()` pose `$dvold = $this`. L'`UPDATE` n'écrit rien. Hydrater depuis `find()`, jamais `new X($id)`. Racine non corrigée.
10	**D15** — `INNER JOIN dvups_lang` après un `LEFT JOIN <entité>_lang`	**non corrigé**	une ligne traduite sans traduction disparaît de **toute** lecture ORM. Contournement : forcer la langue.
11	`get($col, $id_lang)`	**non corrigé**	le second argument est sans effet sur la langue. Utiliser `setLang()` avant.
12	colonne ambiguë sur une entité traduite	**corrigé côté appelants seulement**	`l.name` de `dvups_lang` entre en collision avec la colonne `name` de l'entité. Toujours qualifier `this.<colonne>`.
13	`new Status($id)` puis `->__update()`	**non corrigé**	SQL malformé (`Model.php` + `DBAL.php`). Hydrater par `Status::where(...)->first()` avant.
14	montants en `type="float"`	**non corrigé**	`OrderCore` et les entités d'argent. Arithmétique non exacte.

La règle pratique qui découle des points 2 et 3 : toute valeur venant d'une requête HTTP qui doit entrer dans un `whereIn()` ou un `whereBetween()` doit être validée ou castée **avant**. Le query builder ne le fait pas à votre place sur ces deux méthodes précises — contrairement à `where()`, `whereLike()` et `whereGroup()`.

Fin du Tome II. Le Tome III traite de l'interface : le routage en détail, les contrôleurs, les deux styles d'API, le moteur de listes, le moteur de formulaires, les vues et la sécurité.